

Trilha RTL: Relatório de projeto

Vending Machine

Anderson Manoel de Azevedo Pereira

Mauro José Silva Sousa

Repositório do Projeto: [Acessar código-fonte no GitHub](#)

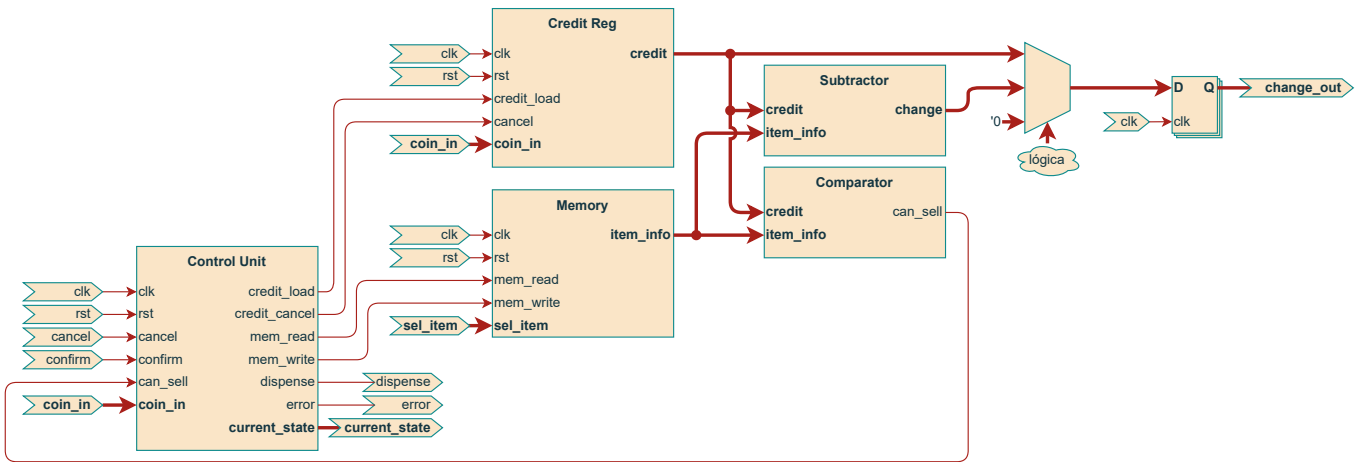
Campina Grande, Paraíba
Julho de 2026

1 Arquitetura do Sistema

1.1 Diagrama de Blocos

Esta seção apresenta o diagrama de blocos do sistema, detalhando as entradas (*coin_in*, *sel_item*, *confirm*, *cancel*, *clk*, *rst*), o registrador de crédito, o comparador, o subtrator e a FSM, conforme as especificações do projeto.

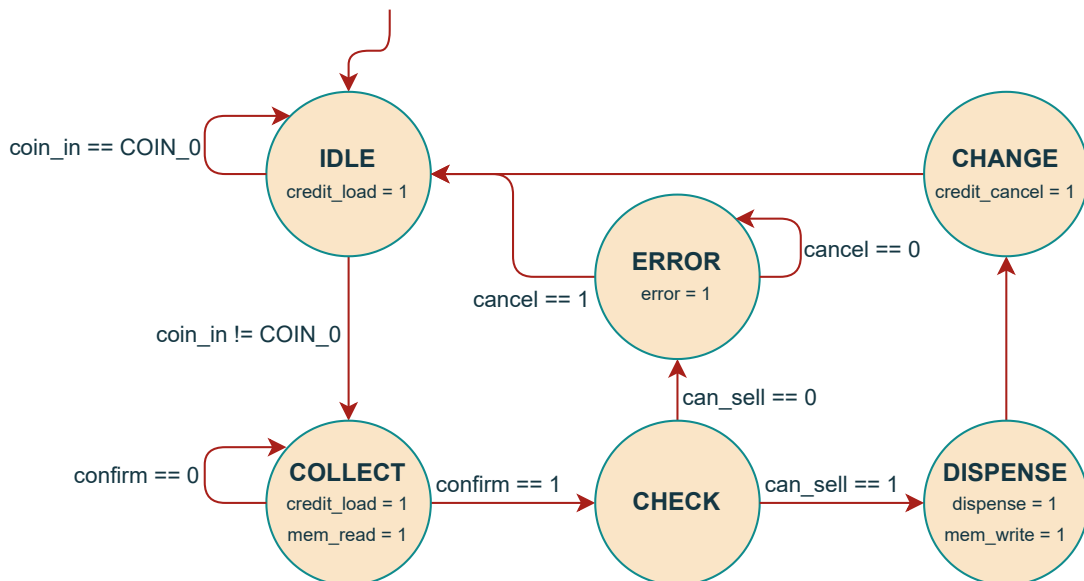
Figura 1 – Diagrama de Blocos da aplicação.



1.2 Diagrama de Estados da FSM

A Figura 2 ilustra a Máquina de Estados Finitos (FSM) de Moore projetada, contendo os 6 estados (IDLE, COLLECT, CHECK, DISPENSE, CHANGE e ERROR), suas transições condicionais e saídas ativas.

Figura 2 – Diagrama de Transição de Estados da Unidade de Controle



2 Descrição dos Módulos

Decisões de projeto e justificativas de cada módulo em SystemVerilog:

- `credit_reg.sv`: registrador síncrono de 8 bits que acumula o valor das moedas inseridas.
- `memory.sv`: memória síncrona 4×16 bits para preço e estoque.
- `comparator.sv`: lógica combinacional do sinal `can_sell`.
- `subtractor.sv`: lógica combinacional do cálculo de `change`.
- `control_unit.sv`: FSM de Moore com 6 estados, orquestra o datapath.
- `vending_top.sv`: top-level, instancia e interconecta os módulos anteriores.

3 Simulação e Validação com Synopsys VCS

3.1 Saída do Testbench

A seguir, o log do terminal demonstrando os resultados (PASS/FAIL) reportados pelo testbench *self-checking*.

```
=====
SCENARIO 1: Successful purchase with change (Coffee)
=====
Buying item: COFFE | Coins inserted: '{COIN_100}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE | Expected: 3, Actual: 3
[PASS] Dispense pulse active | Expected: 1, Actual: 1
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] Change returned: 75 cents | Expected: 75, Actual: 75
[PASS] Return to IDLE | Expected: 0, Actual: 0
[PASS] Credit cleared after sale (display = 0) | Expected: 0, Actual: 0

=====
SCENARIO 2: Insufficient balance (Snack)
=====
Buying item: SNACK | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = ERROR | Expected: 5, Actual: 5
[PASS] Error signal asserted | Expected: 1, Actual: 1
[PASS] Stays in ERROR waiting for cancel | Expected: 5, Actual: 5
[PASS] Return to IDLE after cancel | Expected: 0, Actual: 0

=====
SCENARIO 3: Cancellation and refund
=====
```

```
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] Full refund via change_out: 200 cents | Expected: 200, Actual: 200
[PASS] Display cleared (credit = 0) | Expected: 0, Actual: 0
[PASS] State = IDLE (Immediate return) | Expected: 0, Actual: 0
```

```
=====
SCENARIO 4: Out of stock (Attempt to buy Coffee 6 times)
=====
```

```
--- Coffee purchase attempt 1/5 ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE (In stock) | Expected: 3, Actual: 3
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] State = IDLE | Expected: 0, Actual: 0
--- Coffee purchase attempt 2/5 ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE (In stock) | Expected: 3, Actual: 3
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] State = IDLE | Expected: 0, Actual: 0
--- Coffee purchase attempt 3/5 ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE (In stock) | Expected: 3, Actual: 3
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] State = IDLE | Expected: 0, Actual: 0
--- Coffee purchase attempt 4/5 ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE (In stock) | Expected: 3, Actual: 3
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] State = IDLE | Expected: 0, Actual: 0
--- Coffee purchase attempt 5/5 ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
[PASS] State = DISPENSE (In stock) | Expected: 3, Actual: 3
[PASS] State = CHANGE | Expected: 4, Actual: 4
[PASS] State = IDLE | Expected: 0, Actual: 0
--- Attempt 6 (Stock should be empty) ---
Buying item: COFFE | Coins inserted: '{COIN_25}'
[PASS] State = IDLE | Expected: 0, Actual: 0
[PASS] State = COLLECT | Expected: 1, Actual: 1
[PASS] State = CHECK | Expected: 2, Actual: 2
```

```
[PASS] State = ERROR (Out of stock) | Expected: 5, Actual: 5
[PASS] Error signal asserted due to empty stock | Expected: 1, Actual: 1
[PASS] Final return to IDLE | Expected: 0, Actual: 0
```

```
=====
SIMULATION COMPLETE
=====
```

```
STATUS: PASSED (0 Errors)
```

Log do terminal com os resultados (PASS/FAIL) do testbench self-checking, cobrindo os 4 cenários obrigatórios.

3.2 Waveforms Anotadas

Abaixo estão as capturas de tela do DVE comprovando o funcionamento dos 4 cenários obrigatórios simulados.

Figura 3 – Cenário 1: Compra bem-sucedida.

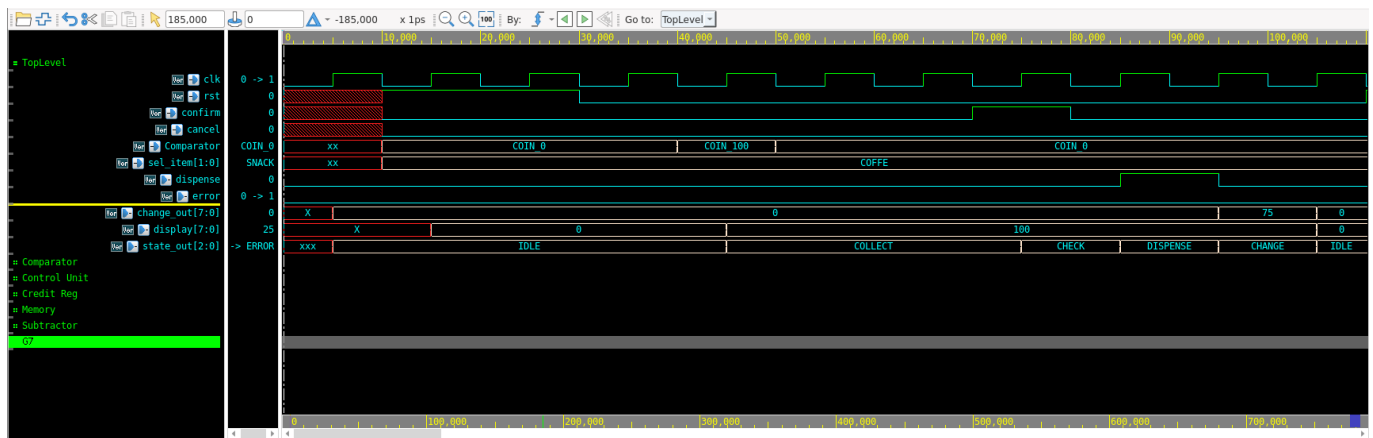


Figura 4 – Cenário 2: Crédito insuficiente.

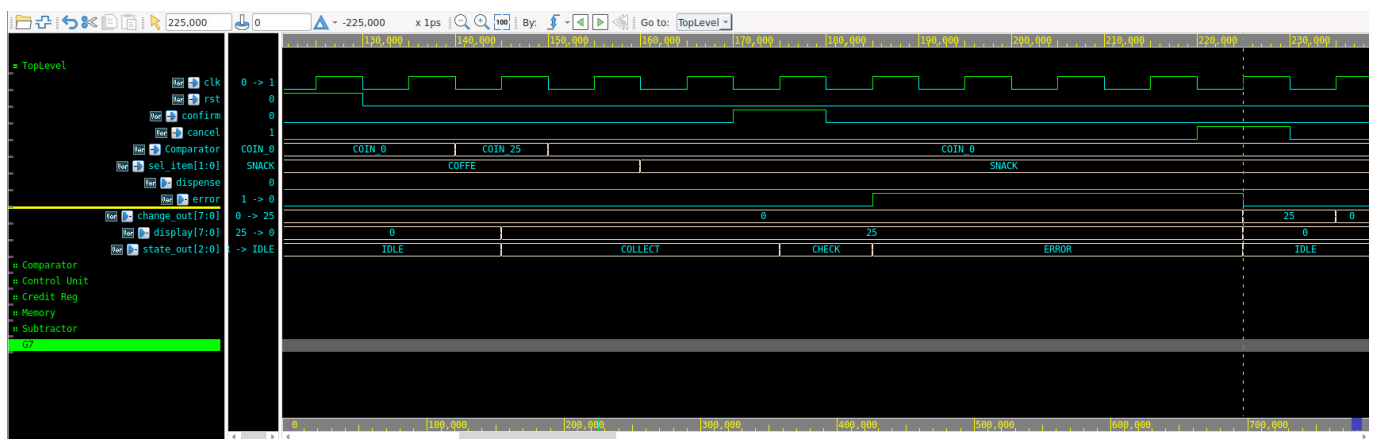


Figura 5 – Cenário 3: Cancelamento.

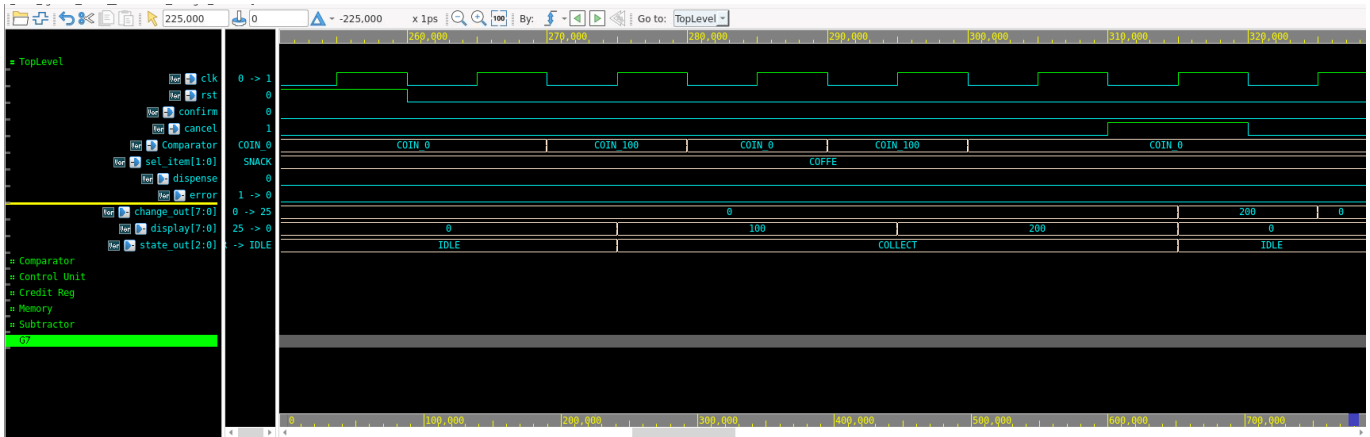
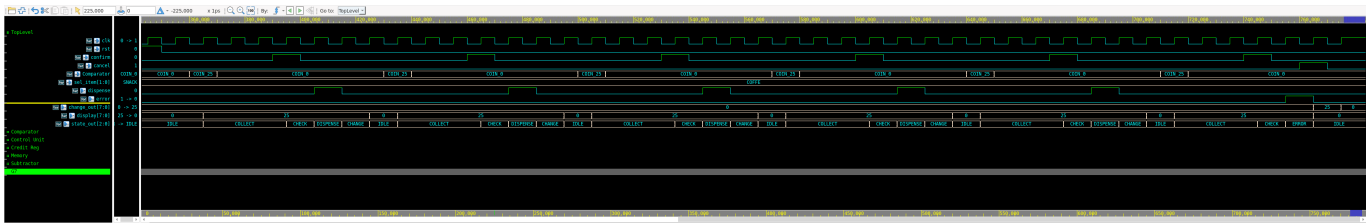


Figura 6 – Cenário 4: Estoque zerado.



4 Síntese e Exploração com Design Compiler

4.1 Resultados da Síntese

A exploração do espaço de projeto gerou os seguintes resultados variando o período do clock em passos de 2 ns, partindo de 20 ns:

Tabela 1 – Exploração de Constraints de Timing e Área

Período (ns)	Frequência (MHz)	Slack (ns)	Área (μm^2)	Timing
20	50,0	17,92	945,42	MET
18	55,6	15,92	945,42	MET
16	62,5	13,92	945,42	MET
10	100,0	7,92	945,42	MET
6	166,7	3,92	945,42	MET
3	333,3	0,92	945,42	MET
2,1	476,2	0,02	945,42	MET
2,1	476,2	0,17	921,02	MET
2,0	500,0	0,01	948,21	MET
2,0	500,0	0,07	921,02	MET
1,9	526,3	0,03	970,07	MET
1,5	666,7	-0,12	1250,39	VIOLATED
1,5	666,7	-0,12	1283,17	VIOLATED

Fonte: Resultados extraídos dos relatórios do Design Compiler. Linhas duplicadas em 2,1 ns, 2,0 ns e 1,5 ns comparam `-no_autoungroup` (primeira linha de cada par) e sem a flag (segunda linha).

A Tabela 1 mostra a variação do período de clock em passos de 2 ns a partir de 20 ns, com refinamento adicional próximo ao limite de timing (2,1 ns a 1,5 ns) para localizar o ponto exato de violação.

4.2 Análise e Discussão do Caminho Crítico

Com base nos relatórios gerados (`report_timing`, `report_area`), discutimos os seguintes pontos requeridos:

1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

Startpoint: `coin_in[1]`. Endpoint: `u_credit_reg/credit_reg[7]/D`. O caminho está inteiramente contido em `credit_reg.sv` e não atravessa `comparator`, `subtractor`, `memory` ou `control_unit`. No período de 2,1 ns (sem `-no_autoungroup`), o atraso total é de 1,82 ns, com 15 portas em série, incluindo três somadores completos (FADDX1) que propagam o carry de `credit + coin_value` até o bit 7.

2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

A área ficou estável em $945,42 \mu m^2$ entre 20 ns e 3 ns, enquanto havia folga de timing. Abaixo de 2,1 ns passou a oscilar ($945,42$ a $970,07 \mu m^2$), à medida que o Design Compiler trocava células por versões de maior *drive strength* para manter o timing fechado. Na primeira rodada violada (1,5 ns), a área subiu para $1250,39$ – $1283,17 \mu m^2$, um aumento de 32–36%, sem que o timing fechasse.

3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?

O limite está entre 2,0 ns (slack positivo, 0,01–0,07 ns) e 1,5 ns (slack de $-0,12$ ns, violado). Após a violação, o slack se estabiliza em $-0,12$ ns mesmo com aumento de área, indicando que a biblioteca `saed32rvt` não oferece célula mais rápida para esse caminho.

4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?

O estado `COLLECT`. É nesse estado que `credit_load` é ativado, disparando `credit <= credit + coin_value`, a operação que gera a cadeia de somadores responsável pelo atraso crítico.

5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

Substituir a soma *ripple-carry* por uma estrutura *carry-lookahead* ou *carry-select*, reduzindo o número de estágios na propagação do carry.

5 Conclusão

O desenvolvimento deste projeto consolidou a aplicação prática de conceitos fundamentais de *RTL Design*. Foi possível vivenciar o ciclo completo do projeto digital desde a modelagem comportamental de uma FSM e sua integração com o *datapath*, passando pela verificação robusta utilizando um testbench *self-checking*, até a etapa de exploração de síntese lógica com o Synopsys Design Compiler.

Um dos principais desafios arquiteturais encontrados residuiu no sincronismo inerente à FSM de Moore. Como as saídas nesse modelo dependem exclusivamente do estado atual, foi necessário antecipar os sinais de controle para alinhar o processamento temporal do *datapath*. Isso ficou evidente na necessidade de iniciar a contagem de crédito logo nas transições a partir do estado `IDLE`, bem como na antecipação da leitura da memória. Esse artefato de projeto garantiu que as informações de preço e estoque já estivessem estabilizadas e disponíveis para o comparador no exato ciclo em que a FSM atingisse o estado `CHECK`.

O roteamento de dados para o mecanismo de troco (`change_out`) também exigiu modelagem cuidadosa. A lógica precisou acomodar o multiplexamento dessa saída, que pode assumir o resultado combinacional do subtrator (crédito - preço) durante uma venda bem-sucedida (estado `CHANGE`), ou o repasse direto do valor integral do registrador de créditos em caso de interrupção pelo sinal de `cancel`.

Na etapa de síntese lógica, um comportamento técnico interessante foi observado nas análises de temporização com a diretiva inicial de incerteza de clock (`set_clock_uncertainty 0.5`) resultou em

violações de *hold time*. Isso ocorreu devido à alta velocidade das portas lógicas na biblioteca tecnológica utilizada (SAED32). Como a correção final de *hold* depende de características físicas da árvore de clock real e da inserção de elementos de atraso (buffers/inversores) na etapa de *Physical Design*, optou-se por focar o esforço de síntese no *setup time*, onde não houve problemas até uma frequência de aproximadamente 500 MHz. Para adequar a análise à realidade do fluxo de *ASIC*, as *constraints* foram corrigidas, adotando valores de incerteza de *hold* típicos para esta biblioteca (0.05 ns).

De forma geral, a principal curva de aprendizado da atividade esteve ligada à interpretação dos relatórios de síntese. Compreender a correlação estrutural entre o caminho crítico, a degradação do *slack* e o impacto na área do circuito sob otimização foi desafiador. O projeto evidenciou que escrever o RTL é apenas o primeiro passo; a exploração minuciosa das restrições de temporização é o que de fato define a viabilidade, a segurança e o desempenho final do hardware.